AI conversations

Well see you have 2 documents firstly Neuroguard-X.pdf which contains the proposed architecture details (GNN + LLM) which will be implemented for the Trojan Detection and Feedback System and other file Implementation.pdf which contains module wise implementation plan for the project, so first be clear about this and ask any question if needed for understanding

Got it — thanks for clarifying. Here's how I understand the setup:

- **NeuroGuard-X.pdf** → This is the **conceptual/proposed architecture** document. It lays out the design of the Trojan Detection and Feedback System, combining **Graph Neural Networks (GNNs)** with **LLMs** for reasoning, bias resistance, and multi-stage detection. Think of this as the *blueprint of what the system should look like*.
- **Implementation.pdf** → This is the **practical, module-wise implementation plan**. It breaks down the project into modules (like Data Ingestion, Graph Construction, etc.), specifies tools (PyVerilog, Yosys, NetworkX, PyTorch Geometric), and outlines step-by-step workflows. This is the *how-to manual for building NeuroGuard-X*.

Before we dive deeper, I want to confirm a couple of things so I can align both documents properly:

1. Do you want me to **map each stage of NeuroGuard-X (architecture)** to the corresponding **modules in**

Implementation.pdf (execution plan)? → Example: Stage 1 (Graph Construction & Feature Engineering in NeuroGuard-X) ↔ Module 1 (Data Ingestion & Graph Construction in Implementation.pdf).

2. Should I focus on **technical alignment** (tools, workflows, data flow) or also include **conceptual reasoning alignment** (why each stage/module exists, how they complement each other)?
3. Are you aiming for a **study/summary document** that integrates both files, or a **project execution roadmap** that shows how to actually build NeuroGuard-X step by step?

Once I know this, I can create a structured mapping that ties the architecture vision to the implementation plan — essentially bridging the "what" and the "how."

Would you like me to start by building that **stage-to-module mapping table**?

so you have understood the 2 files rite?

Yes — I've understood the two files clearly:

- **NeuroGuard-X.pdf** → This is the **proposed architecture** document. It defines the conceptual design of the Trojan Detection and Feedback System, showing how **GNNs + LLMs** are combined in a modular, bias-resistant pipeline. It's the *vision* of the system.
- **Implementation.pdf** → This is the **execution plan**. It breaks the project into modules (like Data Ingestion, Graph

Construction, etc.), specifies tools (PyVerilog, Yosys, NetworkX, PyTorch Geometric), and outlines the step-by-step *workflow* for building NeuroGuard-X.

So in short: one file is the **"what" (architecture)**, the other is the **"how" (implementation plan)**.

Would you like me to now build a **mapping table** that aligns each stage of the NeuroGuard-X architecture with the corresponding implementation modules? That way, you'll have a clear bridge between design and execution.

well so I want you to now focus module wise implementaion of the project : starting from module 1 : firstly tell what is in it and then when I say proceed then provide all the code fragements and requirements for completing the module and repeat this same for all the modules and always wait for me to tell you to proceed

Perfect — let's start module-wise as you requested.

📍 **Module 1: Data Ingestion & Graph Construction**

From **Implementation.pdf**, here's what this module covers:

- **Tools involved:**
  - **PyVerilog** → for parsing RTL/netlist into Abstract Syntax Trees (ASTs).
  - **Yosys** → for synthesis and netlist generation.
  - **NetworkX** → for building structural graphs.

    - **PyTorch Geometric (PyG)** → for handling graph data structures and preparing them for GNN training.

- **Steps outlined:**
    1. **Parse RTL/netlist → AST**
        - Input RTL/netlist files are parsed into an Abstract Syntax Tree using PyVerilog.
    2. **Generate multi-view graphs**
        - Structural graph (connectivity, hierarchy).
        - Functional graph (signal dependencies, logic flow).
        - Possibly semantic/feature graphs (capturing higher-level design attributes).

This module is essentially about **turning raw RTL/netlist code into graph representations** that can later be fed into the GNN pipeline described in NeuroGuard-X.

👉 I'll stop here as per your instruction. When you say **"proceed"**, I'll provide:

- All the **code fragments** (PyVerilog parsing, Yosys integration, NetworkX graph construction, PyG dataset prep).
- The **requirements/environment setup** (Python packages, dependencies, installation commands).
- A **step-wise workflow** to implement Module 1 end-to-end.

Would you like me to proceed with the coding and requirements for Module 1 now?

see this is what my data folder looks like, now proceed according to this

Excellent — now that I've seen your **data folder structure**, I can confirm how Module 1 will operate in your environment.

🧩 Module 1: Data Ingestion & Graph Construction

Your directory layout shows three main datasets:

Folder	Purpose	Contents
TjFree	Clean, Trojan-free Verilog designs	Each subfolder (e.g., *1-Bit Comparator Using Mux*, *4-Bit Multiplier*) contains .v files such as comparator.v, topModule.v.
TRIT-TC	Trojan-infected circuits (Trojan Class)	Subfolders like *c2670_T000* → *c2670_T009* each hold a .v file and a log file documenting Trojan insertion.
TRIT-TS	Trojan-sampled or synthesized test circuits	Likely used for validation and feedback evaluation.

🔍 Role in Module 1

This module's goal is to **parse all Verilog files** from these folders, extract their **ASTs and netlists**, and then **construct graph representations** (nodes = gates/modules, edges = connections). These graphs will later feed into the GNN pipeline for Trojan detection.

🧠 Workflow Overview

1. **Directory traversal** → Identify .v files across TjFree, TRIT-TC, TRIT-TS.
2. **Parsing** → Use **PyVerilog** to convert each file into an AST.
3. **Netlist generation** → Optionally run **Yosys** for structural netlist extraction.
4. **Graph construction** → Build **NetworkX** graphs (connectivity, hierarchy).
5. **Graph dataset preparation** → Convert to **PyTorch Geometric** format for GNN input.

When you say **"proceed,"** I'll provide:

- Complete **code fragments** for each step (PyVerilog parsing, Yosys integration, NetworkX graph creation, PyG dataset conversion).
- **Environment setup** (Python 3.x, required packages, installation commands).
- A **step-wise implementation guide** to execute Module 1 end-to-end on your dataset.

Ready when you are — just say **"proceed."**

⚙️ Environment Setup

First, install the required tools and libraries:

Core parsing and synthesis

pip install pyverilog networkx torch torchvision torchaudio torch-geometric

Yosys (for netlist generation)

sudo apt-get install yosys # Linux

On Windows, use MSYS2 or download binaries from Yosys GitHub

📁 Suggested File Organization

1. file_collector.py
    - Handles scanning your dataset folders (TjFree, TRIT-TC, TRIT-TS).
    - Collects all .v files and returns their paths.
2. verilog_parser.py
    - Uses **PyVerilog** to parse Verilog files into ASTs.
    - Contains functions for extracting modules, ports, and connections.
3. graph_builder.py
    - Builds structural graphs using **NetworkX**.
    - Converts graphs into **PyTorch Geometric** format.
4. main.py
    - The entry point of your project.
    - Calls functions from the above files in sequence:

        - Collect files → Parse → Extract connections → Build graph → Convert to PyG.
    - Prints or saves the resulting graphs.

so give me the workflow without yosys

⚙️ Environment Setup

Install the required Python packages:

bash

```
pip install pyverilog networkx torch torchvision torchaudio torch-geometric
```

📂 Step 1: Collect Verilog Files

Traverse your dataset folders (TjFree, TRIT-TC, TRIT-TS) and gather .v files.

python

```
import os


def collect_verilog_files(base_path):
    verilog_files = []
    for root, dirs, files in os.walk(base_path):
        for f in files:
            if f.endswith(".v"):
                verilog_files.append(os.path.join(root, f))
```

```
    return verilog_files


base_dir = r"D:\Minor_Project\current_work_LLM\current_work\data"
verilog_files = collect_verilog_files(base_dir)
print(f"Found {len(verilog_files)} Verilog files")
```

📄 Step 2: Parse RTL → AST

Use PyVerilog to parse each file into an Abstract Syntax Tree.

```
python
from pyverilog.vparser.parser import parse


def parse_verilog(file_path):
    ast, _ = parse([file_path])
    return ast


ast = parse_verilog(verilog_files[0])
print(ast)
```

🔧 Step 3: Extract Connectivity

Walk the AST to collect module instances and connections.

```
python
from pyverilog.vparser.ast import InstanceList, Instance
```

```
def extract_connections(ast):
    connections = []
    for item in ast.description.definitions:
        for child in item.items:
            if isinstance(child, InstanceList):
                for inst in child.instances:
                    if isinstance(inst, Instance):
                        connections.append((inst.name, inst.module))
    return connections
```

🕸 **Step 4: Build Graph (NetworkX)**

Represent modules and signals as nodes, connections as edges.

```
python
import networkx as nx


def build_graph(connections):
    G = nx.Graph()
    for src, dst in connections:
        G.add_edge(src, dst)
    return G
```

```
connections = extract_connections(ast)
graph = build_graph(connections)
print(nx.info(graph))
```

🧱 **Step 5: Convert to PyTorch Geometric**

Prepare the graph for GNN training.

```
python
import torch
from torch_geometric.data import Data

def nx_to_pyg(G):
    node_map = {n: i for i, n in enumerate(G.nodes())}
    edge_index = torch.tensor(
        [[node_map[u], node_map[v]] for u, v in G.edges()],
        dtype=torch.long
    ).t().contiguous()
    x = torch.ones((len(G.nodes()), 1))  # placeholder features
    return Data(x=x, edge_index=edge_index)

pyg_graph = nx_to_pyg(graph)
print(pyg_graph)
```

✅ **What This Workflow Achieves**

- Directly parses RTL .v files into ASTs.
- Extracts module/gate connectivity without synthesis.
- Builds structural graphs with NetworkX.
- Converts them into PyTorch Geometric datasets for GNN training.

👉 This gives you a **pure-Python workflow** that avoids Yosys entirely. Would you like me to now extend this into a **multi-view graph pipeline** (structural + functional + semantic graphs) so your dataset is richer for Trojan detection?